

# Agentic Services Computing (ASC)

## 1. Introduction & Paradigm Shift

Traditional services computing relied on passive, stateless, function-oriented components (APIs, microservices) executing predictable request-response tasks. The convergence of Large Language Models (LLMs) and Multi-Agent Systems (MAS) has birthed **Agentic Services Computing (ASC)**.

ASC reimagines software services as autonomous, proactive, goal-driven, and socially embedded entities that can perceive, reason, act, and evolve within open and unpredictable environments.

**Core Transition:** Shift from *service invocation* to *agent orchestration*, and from static functional correctness to **behavioral trustworthiness**.

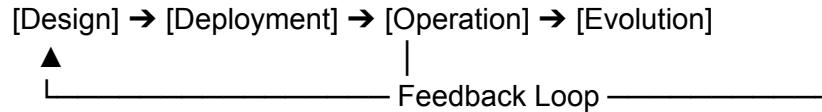
## 2. The SCALE Framework (Key Characteristics)

To distinguish agentic services from traditional web services, the paper introduces the SCALE framework:

- **S – Stateful and Context-Aware:** Maintains persistent long-horizon memory across multiple user sessions via vector databases or knowledge graphs. (Legal Note: This introduces severe challenges regarding data privacy, context boundaries, and compliance regulations like GDPR ).
- **C – Cognitive Autonomy:** Employs LLM-based architectures (e.g., ReAct, Reflexion) for self-correction and continuous agency. This introduces risks of unbounded execution, necessitating safety policy guardrails.
- **A – Agent as Goal Executor:** Rather than executing a static API code string, the agent accepts high-level declarative goals (e.g., "*Prepare a market analysis report*"), decomposes the task into subtasks, and dynamically selects external tools.
- **L – Lifelong Collaborative Learning:** Agents dynamically self-organize, form coalitions, negotiate roles, and distribute tasks using classic multi-agent protocols like the Contract Net Protocol.
- **E – End-to-End Lifecycle Governance:** Mandatory structured tracking across an agent's entire operational lifespan to maintain accountability and minimize liability.

### 3. The 4-Phase Agentic Lifecycle

Unlike static code deployments, ASC enforces a continuous, closed-loop engineering process:



#### A. Design

Establishment of the structural and cognitive blueprint. Designers define explicit organizational topologies, assign agent roles, build safety-by-design filters, and establish model contracts.

#### B. Deployment

Instantiating agents on scalable cloud architecture (Kubernetes, Serverless/FaaS). Crucially, agent artifacts—such as **prompts, system policies, tool bindings, and memory schemas**—must be strictly version-controlled using MLOps primitives to guarantee auditability.

#### C. Operation (*High Relevance to Legal AI*)

Ensures real-time compliance and security under zero-trust principles. Because LLM behaviors are non-deterministic, operation requires **cognitive observability**:

- **Structured Tracing:** Capturing not just system latency, but internal reasoning traces, tool pathways, and multi-agent communications via OpenTelemetry.
- **Human-in-the-Loop (HITL):** Real-time escalation protocols that hand off high-stakes, ambiguous, or ethically critical decisions to a human controller.

#### D. Evolution

The self-improving phase where operational feedback adjusts system logic. Models are refined via alignment techniques like Reinforcement Learning from Human Feedback (RLHF) or Direct Preference Optimization (DPO) to conform to domain-specific norms and values.

## 4. Key Primitives for Trust, Safety, and Value Alignment

For an AI legal department, Section VIII of the paper holds the most critical framework elements, moving the focus from code verification to **behavioral integrity and value alignment**.

| Trust Dimension             | Technical / Governance Primitives                                        | Core Purpose & Function                                                                                                              |
|-----------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Agent Evaluation            | <i>AgentBench, SafeArena, AgentBoard</i>                                 | Shifting evaluation from binary success parameters to tracking process transparency and safety under deliberate misuse scenarios.    |
| Value Alignment             | <i>Constitutional AI, Process Supervision, Cooperative IRL</i>           | Embedding ethical rules directly into reasoning loops so an agent can autonomously reject harmful inputs using a normative rule set. |
| Trustworthiness Engineering | <i>Guardrails AI, Model Cards, Stop Button/Corrigibility, Audit APIs</i> | Establishing machine-verifiable audit trails, enforceable content filters at runtime, and guaranteed shutdown options.               |

### The Emergence of ASLOs

The paper notes that standard corporate SLAs (Service-Level Agreements) are insufficient for autonomous AI. They propose **ASLOs (Agent Service Level Objectives)**. An ASLO legally binds and constrains an agentic service to measurable thresholds regarding **alignment fidelity, interpretability limits, and action boundaries** (e.g., legally mandating that an agent automatically rejects a transaction violating a predefined privacy rule).

## 5. Strategic Legal & Regulatory Frontiers (Future Trends)

The paper identifies specific long-term trajectories where autonomous AI systems intersect directly with institutional frameworks:

- **Regulatory-Aware Agents:** Future agent workflows will require an internal Natural Language Understanding (NLU) layer capable of parsing dynamic legal standards (e.g., GDPR, MiCA EU regulations) and using temporal logic to navigate jurisdictional boundary conflicts automatically.
- **Institutionally Embedded Autonomy:** To safely deploy autonomous ecosystems, agents will require formalized "**Agent Passports**", clear lineage/provenance logs, and standardization across legal architectures to coordinate liability attribution when multi-agent teams fail.
- **Hybrid Legal Frameworks:** The long-term horizon calls for automated systems interacting seamlessly with human legal systems—combining high-speed algorithmic auditing with human judges for dispute resolution and norm enforcement.